

An Introduction to Lean4

Software Verification

MATH230

School of Mathematics and Statistics
University of Canterbury

- ① Software Verification
- ② List
- ③ Software Verification in Practice

Software Specifications

Lean is first and foremost a programming language. Which means we can write the programs we would normally write; as well as programs that are being used as proofs.

In fact, we can write programs as we normally would and then, all within Lean, prove that they meet the specifications we expect.

Previously we proved theorems about addition and multiplication on the natural numbers. These are just statements about how programs on the data type $\mathbb{N}$ interact. There is no reason why we can't write different programs on different data types and prove properties about those programs.

First order logic (aka dependent types) can be used to write formal specifications for software!

This is not unit testing with cases.

Lists can be created in two ways; stating the empty list, or cons-ing an element to the beginning of another list.

```
inductive List ( $\alpha$  : Type) where  
  | null : List  $\alpha$   
  | cons : ( $a$  :  $\alpha$ )  $\rightarrow$  ( $as$  : List  $\alpha$ )  $\rightarrow$  List  $\alpha$ 
```

This type has a very similar structure to the type of natural numbers. Writing functions out of this type will also be done by pattern matching; accounting for the null case, and the cons case.

Append is a function which takes two lists and concatenates them into a single list; the first onto the beginning of the second. Here is a recursive definition of append using pattern matching on the first argument:

```
def append { $\alpha$  : Type} (as bs : List  $\alpha$ ) : List  $\alpha$  :=  
  match as with  
  | null      => bs  
  | cons a as => cons a (append as bs)
```

Specifications for Append

This program should have each of the following properties:

```
def null_append { $\alpha$  : Type} :  
   $\forall$  xs : List  $\alpha$ , (null ++ xs) = xs
```

```
def cons_append { $\alpha$  : Type} :  
   $\forall$  x :  $\alpha$ ,  $\forall$  xs ys : List  $\alpha$ ,  
    ((cons x xs) ++ ys) = (cons x (xs ++ ys))
```

```
theorem append_null { $\alpha$  : Type} :  
   $\forall$  xs : List  $\alpha$ , (xs ++ null) = xs
```

```
theorem append_assoc { $\alpha$  : Type} :  
   $\forall$  xs ys zs : List  $\alpha$ , (xs ++ ys) ++ zs = xs ++ (ys ++ zs)
```

```
theorem length_append { $\alpha$  : Type} :  
   $\forall$  xs ys : List  $\alpha$ , length (xs ++ ys) = length xs + length ys
```

Definitionally Equal

These are definitionally equal!

```
def null_append { $\alpha$  : Type} :  
   $\forall$  xs : List  $\alpha$ , (null ++ xs) = xs
```

```
def cons_append { $\alpha$  : Type} :  
   $\forall$  x :  $\alpha$ ,  $\forall$  xs ys : List  $\alpha$ ,  
    ((cons x xs) ++ ys) = (cons x (xs ++ ys))
```

Pen-and-Paper Proof

What would a proof of a statement like this even look like?

```
theorem append_null { $\alpha$  : Type} :  
   $\forall$  xs : List  $\alpha$ , (xs ++ null) = xs
```


Induction on Lists

This suggests that we can do induction on lists; which shouldn't be too suprising given that the type `List  $\alpha$` is very similar to $\mathbb{N}$ the type of natural numbers.

```
theorem append_null { $\alpha$  : Type} :  
   $\forall$  xs : List  $\alpha$ , (xs ++ null) = xs :=  
  by  
    intro as  
    induction as with  
    | null          => _  
    | cons k ks ih => _
```

Pen-and-Paper Proof

```
theorem append_assoc {α : Type} :  
  ∀ xs ys zs : List α, (xs ++ ys) ++ zs = xs ++ (ys ++ zs)
```

Pen-and-Paper Proof

```
def length { $\alpha$  : Type} : List  $\alpha$  → Nat
| null      => 0
| cons _ as => 1 + length as
```

```
theorem length_append { $\alpha$  : Type} :
   $\forall$  xs ys : List  $\alpha$ , length (xs ++ ys) = length xs + length ys
```

Software Verification

With these examples we begin to see how properties of programs can be expressed in the dependent type system of Lean4. Moreover, we can write programs that inhabit these properties/propositions in order to prove them. In this way we are able to verify programs meet certain specifications *for all possible inputs*.

Example: One can implement `merge_sort` in Lean4 and also write a specification to say what it means for a list to be sorted. This would be a unary predicate, `Sorted`, on `List`. Finally, one could prove the theorem:

```
theorem merge_sorted {α : Type} :  
  ∀ xs : List α, Sorted (merge_sort xs)
```

Programming and verification all happen in the same environment.

Verification Examples

As we have seen, formally verifying that proofs/programs meet specifications can be a lot of work. For this reason the security pay-off has to be important enough to invest time (and money) into the process. Here are just a few examples where formal verification of the type we've studied has been used in practice.

- CompCert C Compiler (Rocq)

- Cardano proof-of-stake protocol (Agda and HOL)

- sel4 OS kernel verification project (HOL)

- HACL verified Cryptography Library (F*)

You can click on each of these to find more information about the projects.